ISOM 671

Managing Big Data

RAJIV GARG

Associate Professor

Information Systems & Operations Management

September 9, 2024

EMORY

GOIZUETA BUSINESS SCHOOL

Expectations!

- **Stay safe and keep others safe:**
  - If you have covid like symptoms, test negative, and are attending class, please maintain distance for safety of others.
- **Discussion:**
  - Raise hand to ask any questions
  - We may have small group discussions.
- **Quiz:**
  - There may be a 5-minute quiz before class on some days, and it will be administered on Canvas (no paper). You need to make sure your canvas account is accessible during class time.
- **Be courteous:**
  - I don’t mind if you are speaking to someone else or doing something else on your computer as long as you are not distracting anyone else in the class. If you are the reason for distraction, I will have to ask you to leave the class.
- **PLEASE USE NAMECARDS**

Assignment 1 (3%): Due 9/12 (by 8:30am)

- Get NYC Airbnb data from Canvas: airbnb_nyc_clean.csv
  - Source: Kaggle (https://www.kaggle.com/datasets/arianazmoudeh/airbnbopendata)
    - Alt: https://www.kaggle.com/datasets/sandeepmajumdar/airbnbnyccleanedLinks to an external site.
-
- Submission:
  - Design a database:
    - Import data in mySQL
    - Follow IDTKNI process to identify 3NF normalized tables
      - Identify data-elements, Divide data-elements, Identify Tables, Identify Keys, Normalize, Develop Index
    - Create normalized tables in mySQL and populate with data
    - Submit the first 5 rows of each table
  - Develop a Conceptual Design of the DB
    - Create all relationships between tables (you may need to create new keys)
    - Reverse engineer to get the ER Model of normalized data
    - Submit the ER model

Catchup

Normalization

SQL I

Structured Query Language

September 9, 2024

EMORY
GOIZUETA
BUSINESS
SCHOOL

SQL

- What is SQL?
  - Higher level language to query, modify, and create databases
  - Resulted from relational database model proposed by Edgar Codd (IBM researcher) in 1969

- Why SQL?
  - Bigger (can handle increasing volume of data – both structured and NoSQL)
  - Faster (binary storage, file splitting, indexing)
  - Cheaper (open source, ODBC drivers, platform agnostic)

SQL

- Structured Query Language (SQL) is a standard language for relational database management systems (RDBMS). It specifies the syntax for data definition and manipulation:

- Data Definition Language (DDL)
  - Defines the schema
    - Table – CREATE, ALTER, DROP
    - User – ADD, ALTER, DROP
    - View – CREATE
    - Index – CREATE
    - CONSTRAINT (primary key, unique key)

- Data Manipulation Language (DML)
  - Modifies and queries the data
    - Data – INSERT INTO, DELETE, UPDATE
    - SELECT
    - JOIN – inner, left, right

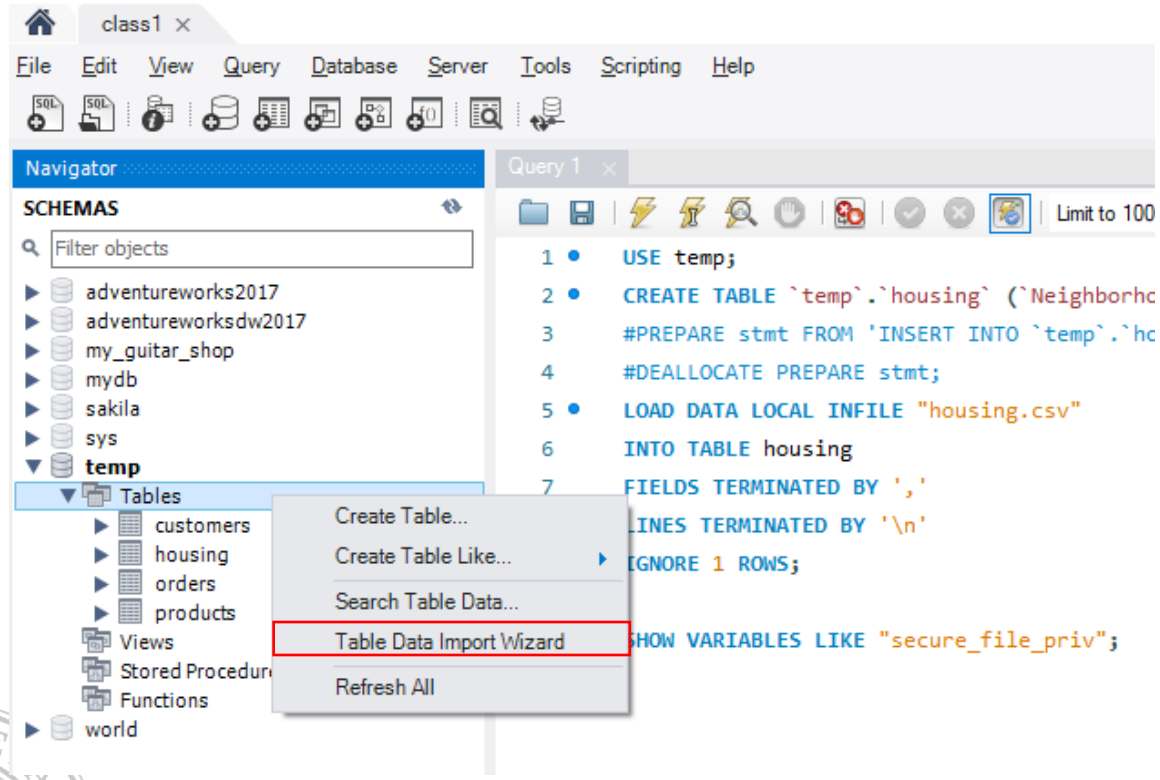
For more information: https://docs.microsoft.com/en-us/office/client-developer/access/desktop-database-reference/microsoft-access-sql-reference

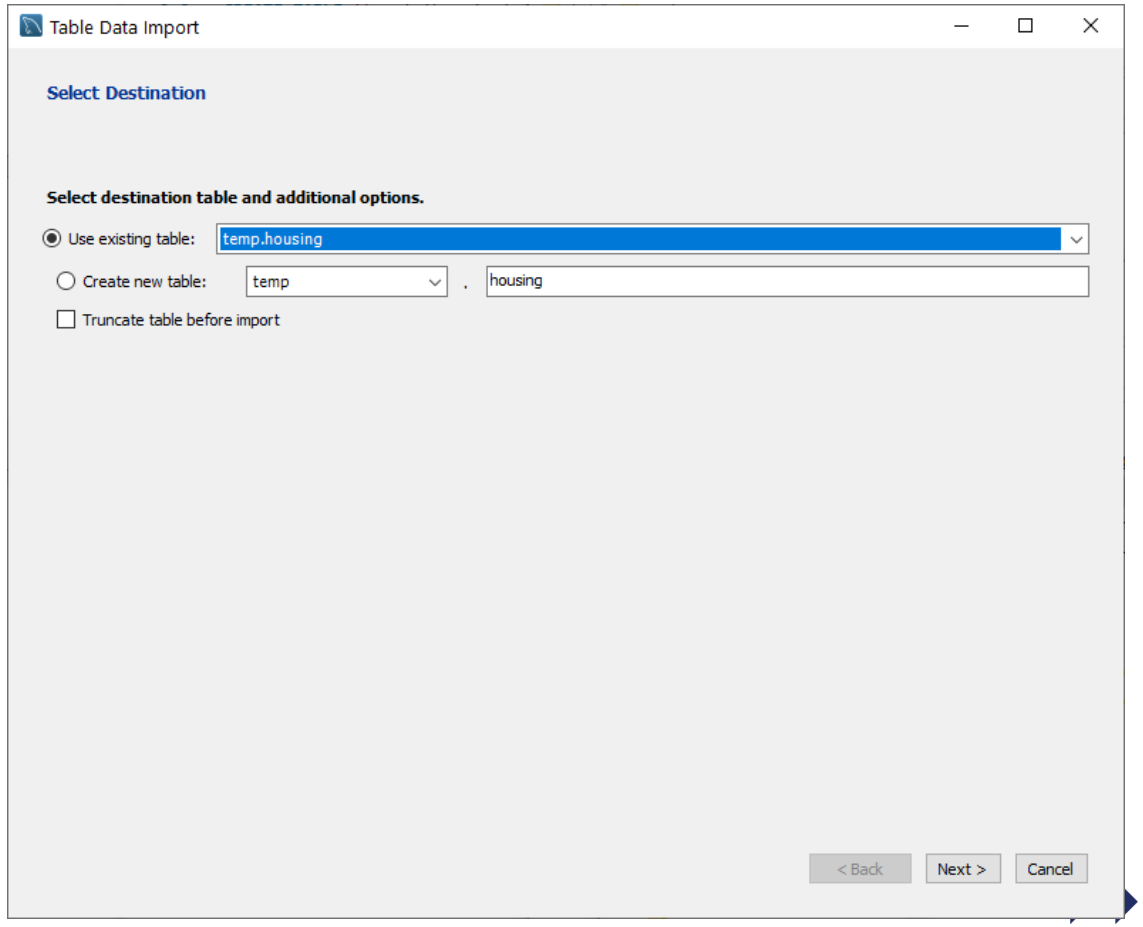
SQL: Add Data

- Load from external source:
  - Import (e.g., csv, excel, etc.)

- Programmatically (e.g., in mySQL, Python, Excel, etc.):
  - Using DB software/command-line:
    - Create table
    - Inset data
  - Using ODBC and other connecting applications

SQL: Load Data (csv in mySQL)





SQL - CREATE TABLE

```
CREATE TABLE
    { database_name.schema_name.table_name | schema_name.table_name | table_name }
    ({   <column_definition>
        | <computed_column_definition>
        | <column_set_definition>
        | [ <table_constraint> ] [ ,... n ]
        | [ <table_index> ] }
       [ ,...n ] )
[ ; ]
```

More details: https://docs.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql?view=sql-server-ver15

SQL - CREATE TABLE

```
CREATE TABLE customers (
customer_id INT,
email_address CHAR(64),
password CHAR(64),
fName CHAR(32),
lName CHAR(32),
address_billing INT,
address_shipping INT,
CONSTRAINT pk_customer PRIMARY KEY (customer_id));
```

Primary Key

SQL - CREATE TABLE

```
CREATE TABLE products (
  product_id int NOT NULL AUTO_INCREMENT,
  category_id int,
  product_code text,
  product_name char(64),
  list_price float,
  discount_percent float,
  date_added datetime,
CONSTRAINT pk_products PRIMARY KEY (product_id));
```

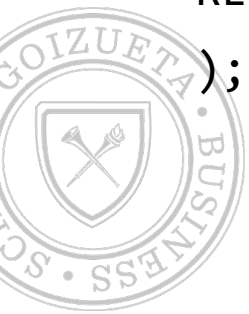
SQL - CREATE TABLE

```
CREATE TABLE orders (
customer_id INT,
product_id INT,
card_type ENUM('Visa', 'Master', 'Amex'),
CONSTRAINT pk_order PRIMARY KEY (customer_id, product_id),
CONSTRAINT fk_customer FOREIGN KEY (customer_id)
REFERENCES customers (customer_id),
CONSTRAINT fk_product FOREIGN KEY (product_id)
REFERENCES products (product_id)
);
```

Would be **composite key** if these were not foreign keys

Compound Key

CONSTRAINT for FK needs REFERENCES



SQL – DATA TYPES

String data types:

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The *size* parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The *size* parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The *size* parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The *size* parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large OBjects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large OBjects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large OBjects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LONGBLOB	For BLOBs (Binary Large OBjects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Numeric data types:

Data type	Description
BIT(*size*)	A bit-value type. The number of bits per value is specified in *size*. The *size* parameter can hold a value from 1 to 64. The default value for *size* is 1.
TINYINT(*size*)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The *size* parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(*size*)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The *size* parameter specifies the maximum display width (which is 255)
MEDIUMINT(*size*)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The *size* parameter specifies the maximum display width (which is 255)
INT(*size*)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The *size* parameter specifies the maximum display width (which is 255)
INTEGER(*size*)	Equal to INT(size)
BIGINT(*size*)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The *size* parameter specifies the maximum display width (which is 255)
FLOAT(*size*, *d*)	A floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(*p*)	A floating point number. MySQL uses the *p* value to determine whether to use FLOAT or DOUBLE for the resulting data type. If *p* is from 0 to 24, the data type becomes FLOAT(). If *p* is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(*size*, *d*)	A normal-size floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter
DOUBLE PRECISION(*size*, *d*)	
DECIMAL(*size*, *d*)	An exact fixed-point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. The maximum number for *size* is 65. The maximum number for *d* is 30. The default value for *size* is 10. The default value for *d* is 0.

Date and Time data types:

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(*fsp*)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(*fsp*)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(*fsp*)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

Source: https://www.w3schools.com/sql/sql_datatypes.asp

SQL – LOAD DATA

- LOAD DATA INFILE 'data.csv' INTO TABLE dbname.my_table;
  - May give security error (need to edit secure-file-priv setting in **my.ini** file)

OR

- LOAD DATA INFILE 'data.csv' INTO TABLE dbname.my_table;
  - May give restriction error (need to add **OPT_LOCAL_INFILE=1** in advanced connection settings)

- You may also be required to specify file structure (separator and eol character):
  - LOAD DATA INFILE 'data.csv' INTO TABLE dbname.my_table
    FIELDS TERMINATED BY "," LINES TERMINATED BY "\n";

More details: https://dev.mysql.com/doc/refman/8.0/en/load-data.html

SQL – DROP

- DROP database:

- DROP TABLE customers;

SQL – ALTER TABLE

- ALTER TABLE table {ADD {COLUMN field type[(size)] [NOT NULL] [CONSTRAINT index] | ALTER COLUMN field type[(size)] | CONSTRAINT multifieldindex} | DROP {COLUMN field I CONSTRAINT indexname} }

- **ALTER TABLE customer ADD COLUMN Notes TEXT;**
- INSERT INTO customer (`Notes`) VALUES ('52');
- ALTER TABLE customer **MODIFY** Notes INT;
- **ALTER TABLE customer DROP COLUMN Notes;**

SQL – ALTER TABLE (ADD Keys)

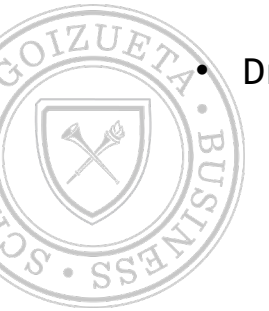
- Import CSV data files using wizard

- Create primary keys:
  - ALTER TABLE customer ADD **CONSTRAINT pk_customer** PRIMARY KEY(customer_id);
  - ALTER TABLE product ADD PRIMARY KEY(product_id);
  - ALTER TABLE sales ADD PRIMARY KEY(transaction_id);

"CONSTRAINT pk_customer" is optional (by default, PRIMARY KEY is a constraint)

- Create foreign keys:
  - ALTER TABLE inventory ADD CONSTRAINT fk_inv_pid FOREIGN KEY (product_id) REFERENCES product(product_id);
  - ALTER TABLE sales ADD CONSTRAINT fk_pid FOREIGN KEY (product_id) REFERENCES product(product_id);
  - ALTER TABLE sales ADD CONSTRAINT fk_cid FOREIGN KEY (customer_id) REFERENCES customer(customer_id);

- Drop keys:
  - ALTER TABLE product **DROP PRIMARY KEY**;
  - ALTER TABLE orders **DROP CONSTRAINT** fk_product;



SQL – TABLE INFORMATION

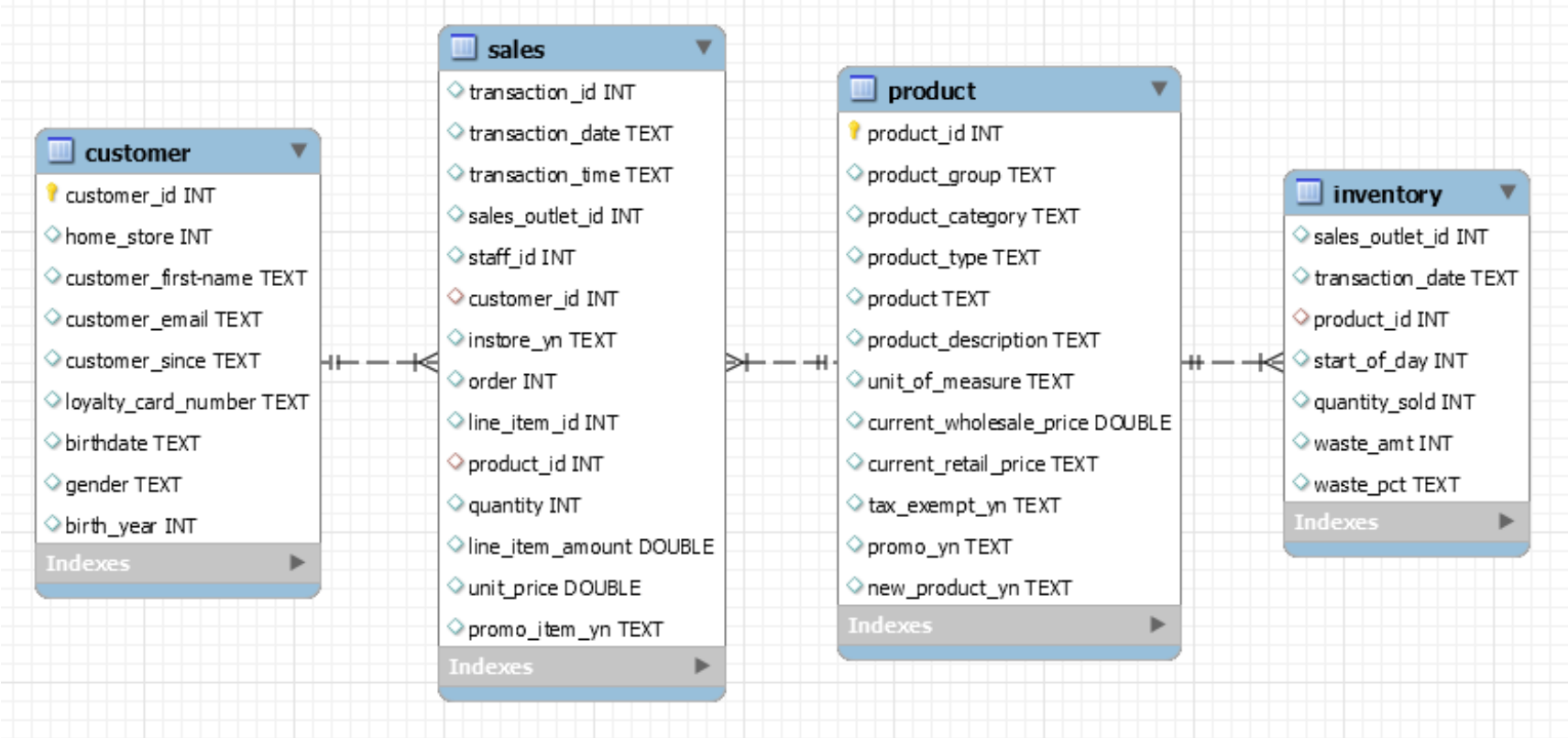
- The TABLE_CONSTRAINTS table describes which tables have constraints.

- SELECT *
- FROM information_schema.table_constraints
- WHERE table_schema="isom671";

CONSTRAINT_CATALOG	CONSTRAINT_SCHEMA	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE	ENFORCED
def	isom671	PRIMARY	isom671	customers	PRIMARY KEY	YES
def	isom671	PRIMARY	isom671	orders	PRIMARY KEY	YES
def	isom671	fk_customer	isom671	orders	FOREIGN KEY	YES
def	isom671	fk_product	isom671	orders	FOREIGN KEY	YES
def	isom671	PRIMARY	isom671	product	PRIMARY KEY	YES
def	isom671	PRIMARY	isom671	products	PRIMARY KEY	YES

- Ref:
  - https://dev.mysql.com/doc/refman/8.0/en/information-schema-introduction.html
  - https://dev.mysql.com/doc/refman/8.0/en/information-schema-table-constraints-table.html

Schema - Coffee Shop



SQL - DML

INSERT, DELETE, UPDATE

SQL - INSERT (new rows)

- You can use the INSERT statement to add one or more rows to a table.
- In the INSERT clause, you specify the name of the table that you want to add a row to, along with an optional column list.
  - The INTO keyword is also optional.
- In the VALUES clause, you specify the values to be inserted.
- If you don’t include a column list in the INSERT clause, you must specify the column values in the same order as in the table, and you must code a Value for each column.

SQL – INSERT Data

- INSERT INTO customers VALUES (1,'allan.sherwood@yahoo.com','650215acec746f0e32bdfff387439eefc1358737','Allan','Sherwood',1,2);

- INSERT INTO customers VALUES (2,'barryz@gmail.com','3f563468d42a448cb1e56924529f6e7bbe529cc7','Barry','Zimmer',3,3);

- INSERT INTO customers VALUES (3,'christineb@solarone.com','ed19f5c0833094026a2f1e9e6f08a35d26037066','Christine','Brown',4,4)

SQL - INSERT (new rows) Examples

The syntax of the INSERT statement

```
INSERT [INTO] table_name [(column_list)]
VALUES (expression_1[, expression_2]...)[,
       (expression_1[, expression_2]...)]...
```

The column definitions for the Invoices table

```
invoice_id          INT             NOT NULL     AUTO_INCREMENT,
vendor_id           INT             NOT NULL,
invoice_number      VARCHAR(50)     NOT NULL,
invoice_date        DATE            NOT NULL,
invoice_total       DECIMAL(9,2)    NOT NULL,
payment_total       DECIMAL(9,2)    NOT NULL     DEFAULT 0,
credit_total        DECIMAL(9,2)    NOT NULL     DEFAULT 0,
terms_id            INT             NOT NULL,
invoice_due_date    DATE            NOT NULL,
payment_date        DATE
```

Insert a single row without using a column list

```
INSERT INTO invoices VALUES
(115, 97, '456789', '2011-08-01', 8344.50, 0, 0, 1, '2011-08-31', NULL)

(1 row affected)
```

Insert a single row using a column list

```
INSERT INTO invoices
    (vendor_id, invoice_number, invoice_total, terms_id, invoice_date,
     invoice_due_date)
VALUES
    (97, '456789', 8344.50, 1, '2011-08-01', '2011-08-31')

(1 row affected)
```

Insert multiple rows

```
INSERT INTO invoices VALUES
(116, 97, '456701', '2011-08-02', 270.50, 0, 0, 1, '2011-09-01', NULL),
(117, 97, '456791', '2011-08-03', 4390.00, 0, 0, 1, '2011-09-02', NULL),
(118, 97, '456792', '2011-08-03', 565.60, 0, 0, 1, '2011-09-02', NULL)
```

SQL - INSERT (NULL and DEFAULT)

- If a column is defined so it allows null values, you can use the **NULL** keyword in the list of values to insert a null value into that column.

- If a column is defined with a default (or auto increment) value, you can use the **DEFAULT** keyword in the list of values to insert the default (or generate the next) value for that column.

- If you include a column list, you can omit columns with default values, auto increment and null values. Then, the corresponding value is assigned automatically.

SQL - INSERT (NULL and DEFAULT)

The column definitions for the Color_Sample table

```
color_id        INT             NOT NULL     AUTO_INCREMENT,
color_number    INT             NOT NULL     DEFAULT 0,
color_name      VARCHAR(50)
```

Five INSERT statements for the Color_Sample table

```
INSERT INTO color_sample (color_number)
VALUES (606)

INSERT INTO color_sample (color_name)
VALUES ('Yellow')

INSERT INTO color_sample
VALUES (DEFAULT, DEFAULT, 'Orange')

INSERT INTO color_sample
VALUES (DEFAULT, 808, NULL)

INSERT INTO color_sample
VALUES (DEFAULT, DEFAULT, NULL)
```

The Color_Sample table after the rows have been inserted

color_id	color_number	color_name
1	606	NULL
2	0	Yellow
3	0	Orange
4	808	NULL
5	0	NULL

SQL - INSERT (subquery)

- To insert rows selected from one or more tables into another table, you can code a subquery in place of the VALUES clause.
    - Then, MySQL inserts the rows returned by the subquery into the target table. For this to work, the target table must already exist.

- The rules for working with a column list are the same as they are for any INSERT statement.

SQL - INSERT (subquery)

The syntax for using a subquery to insert one or more rows

```
INSERT [INTO] table_name [(column_list)] select_statement
```

Insert paid invoices into the Invoice_Archive table

```
INSERT INTO invoice_archive
SELECT *
FROM invoices
WHERE invoice_total - payment_total - credit_total = 0

(103 rows affected)
```

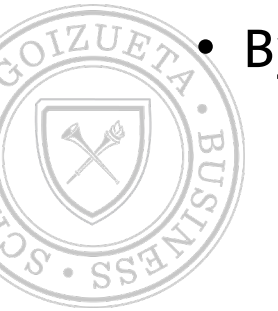
The same statement with a column list

```
INSERT INTO invoice_archive
    (invoice_id, vendor_id, invoice_number, invoice_total, credit_total,
    payment_total, terms_id, invoice_date, invoice_due_date)
SELECT
    invoice_id, vendor_id, invoice_number, invoice_total, credit_total,
    payment_total, terms_id, invoice_date, invoice_due_date
FROM invoices
WHERE invoice_total - payment_total - credit_total = 0

(103 rows affected)
```

SQL - UPDATE (existing rows)

- You can use the UPDATE statement to modify one or more rows in a table.
- In the SET clause, you name each column and its new value.
  - You can specify the value for a column as a literal or an expression.
  - You can also use the DEFAULT and NULL keywords to specify default and null values.
- In the WHERE clause, you can specify the conditions that must be met for a row to be updated.
- By default, MySQL Workbench runs in safe update mode.
  - That prevents you from updating rows if the WHERE clause is omitted or doesn't refer to a primary key or foreign key column.



SQL - UPDATE (existing rows)

The syntax of the UPDATE statement

```
UPDATE table_name
SET column_name_1 = expression_1[, column_name_2 = expression_2]...
[WHERE search_condition]
```

Update two columns for a single row

```
UPDATE invoices
SET payment_date = '2011-09-21',
    payment_total = 19351.18
WHERE invoice_number = '97/522'

(1 row affected)
```

Update one column for multiple rows

```
UPDATE invoices
SET terms_id = 1
WHERE vendor_id = 95

(6 rows affected)
```

Update one column for one row

```
UPDATE invoices
SET credit_total = credit_total + 100
WHERE invoice_number = '97/522'

(1 row affected)
```

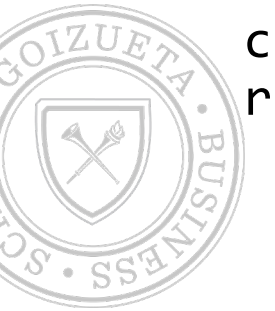
SQL – UPDATE Data

- UPDATE customers SET lName = 'Green' WHERE fName = 'Christine’;

- Note: Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a KEY column. To disable safe mode, toggle the option in **Preferences -> SQL Editor** and reconnect.

SQL - DELETE (existing rows)

- You can use the DELETE statement to delete one or more rows from the table you name in the DELETE clause.
- You specify the conditions that must be met for a row to be deleted in the WHERE clause.
- You can use a subquery within the WHERE clause.
- A foreign-key constraint may prevent you from deleting a row. In that case, you can only delete the row if you delete all child rows for that row first.



SQL - DELETE (existing rows)

The syntax of the DELETE statement

```
DELETE FROM table_name
[WHERE search_condition]
```

Delete one row

```
DELETE FROM general_ledger_accounts
WHERE account_number = 306

(1 row affected)
```

Delete one row using a compound condition

```
DELETE FROM invoice_line_items
WHERE invoice_id = 78 AND invoice_sequence = 2

(1 row affected)
```

Delete multiple rows

```
DELETE FROM invoice_line_items
WHERE invoice_id = 12

(4 rows affected)
```

Use a subquery in a DELETE statement

```
DELETE FROM invoice_line_items
WHERE invoice_id IN
    (SELECT invoice_id
     FROM invoices
     WHERE vendor_id = 115)

(4 rows affected)
```

SQL - DELETE (existing rows)

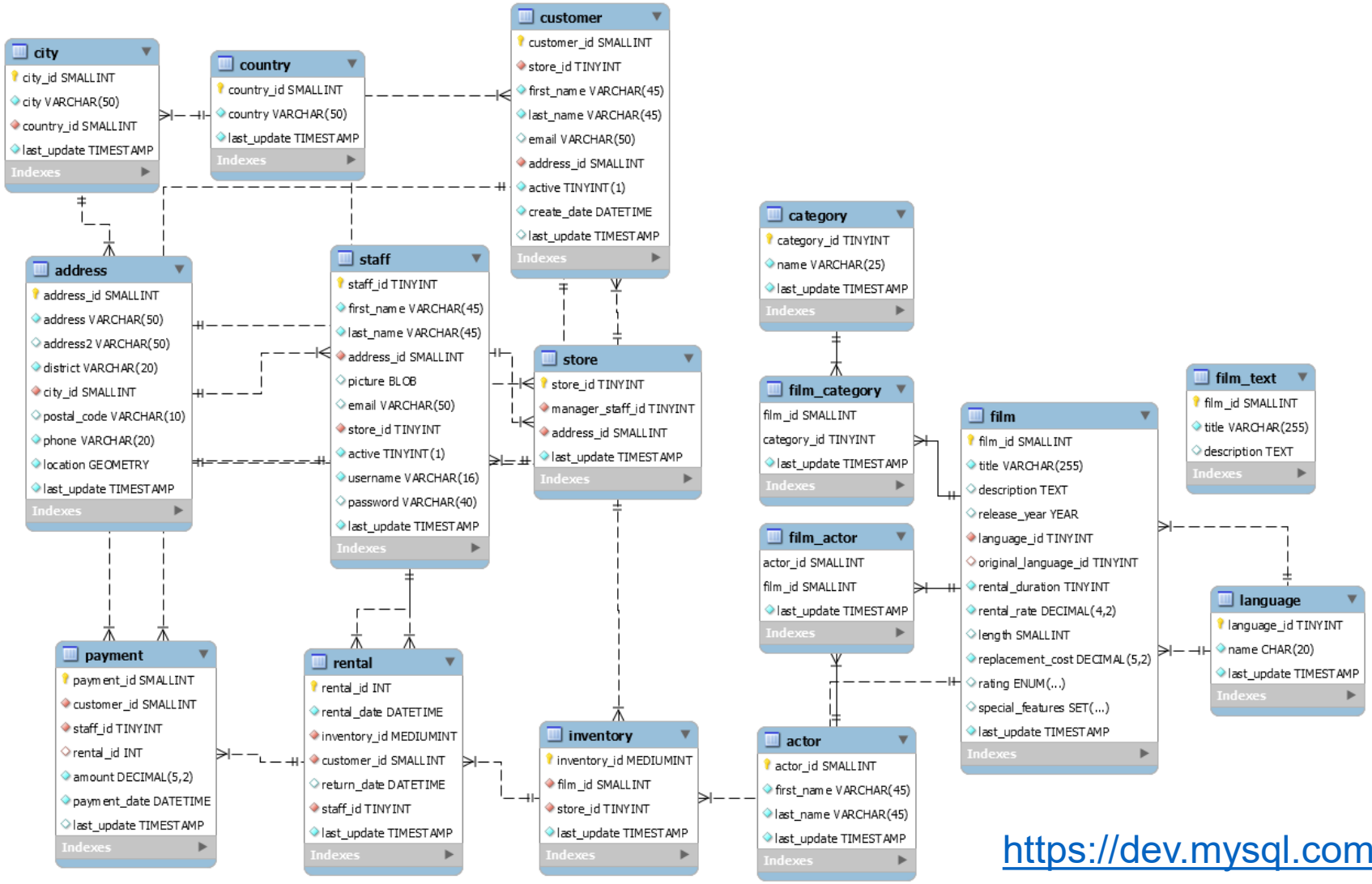
- DELETE FROM customers WHERE fName = 'Barry’

- Note: Error 1175 in mySQL implies that you can’t delete without using primary key, or you need to turn off safe mode (SET SQL_SAFE_UPDATES = 0;).

SQL - SELECT

Sakila

ER Model - sakila



https://dev.mysql.com/doc/sakila/en/

Sakila - SQL Create Trigger Error

- Import movie rental (Sakila) database:
  - Source: https://dev.mysql.com/doc/sakila/en/

- Possible trigger creation error when loading on AWS RDS:
  - CREATE TRIGGER `ins_film` AFTER INSERT ON `film` FOR EACH ROW BEGIN
  - INSERT INTO film_text (film_id, title, description)
  - VALUES (new.film_id, new.title, new.description);
  - END;

- The CREATE TRIGGER statement allows you to create a trigger that is executed automatically whenever an event (INSERT, DELETE, or UPDATE) occurs against a table.
  - To create such automatically executing events, you need to enable the setting on the SQL server. By default, the setting is set to “0” on AWS RDS.
  - You can change it on AWS by following the steps described here:
    - https://aws.amazon.com/premiumsupport/knowledge-center/rds-mysql-functions/

SQL - SELECT

- SELECT [predicate] { * | table.* | [table.]field1 [AS alias1] [, [table.]field2 [AS alias2] [, …]]} FROM tableexpression [, …] [IN externaldatabase] [WHERE…] [GROUP BY…] [HAVING…] [ORDER BY…] [WITH OWNERACCESS OPTION]

- SELECT Execution Order:
  - FROM – indicates the table from which the data is selected
  - WHERE - indicates conditions under which a row will be included in the result
  - GROUP BY - indicates categorization of results
  - HAVING - indicates the conditions under which a group/category will be included
  - SELECT - columns to be returned
  - ORDER BY – sort results

  - [output]

SQL – SELECT

- Show movie title, description, release year and rating:

```
SELECT title, description, release_year, rating FROM sakila.film;
```

- List all unique customer locations (district) in address table

```
SELECT DISTINCT district  FROM sakila.address;
```

- List all movie names and rental prices in ascending order:

```
SELECT title, release_year, rental_rate FROM sakila.film ORDER BY rental_rate ASC;
```

SQL – SELECT

- Show release years (in ascending order) of movies in the data:

```
SELECT release_year FROM sakila.film ORDER BY release_year;
```

- Show unique release years (in ascending order) of movies in the data:

```
SELECT DISTINCT release_year FROM sakila.film ORDER BY release_year;
```

- Show unique rental rate and release year combinations (in ascending order of rental rate) of movies in the data:

```
SELECT DISTINCT release_year, rental_rate FROM sakila.film ORDER BY rental_rate;
```

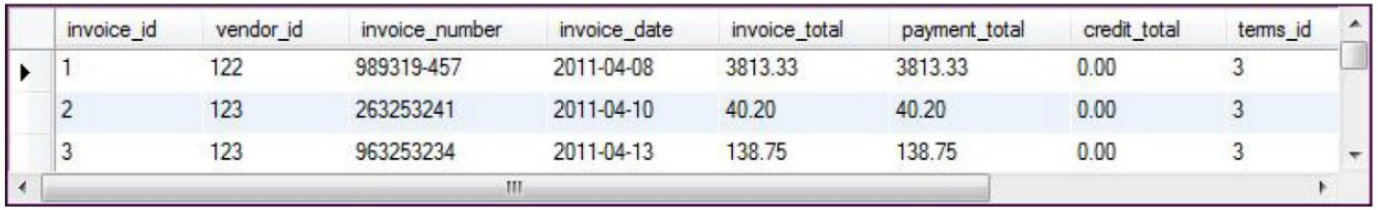
ORDER BY clause

- The ORDER BY clause specifies how you want the rows in the result set sorted. You can sort by one or more columns, and you can sort each column in either ascending (ASC) or descending (DESC) sequence.
    - ASC is the default.
- In an ascending sort, special characters appear first in the sort sequence, followed by numbers, then letters.
    - This sort order is determined by the character set used by the server.
- Null values appear first in the sort sequence, even if you’re using DESC.
- You can sort by any column in the base table regardless of whether it’s included in the SELECT clause.

SQL – SELECT (example)

A SELECT statement that retrieves all the data from the Invoices table

```
SELECT * FROM invoices
```



invoice_id	vendor_id	invoice_number	invoice_date	invoice_total	payment_total	credit_total	terms_id
1	122	989319-457	2011-04-08	3813.33	3813.33	0.00	3
2	123	263253241	2011-04-10	40.20	40.20	0.00	3
3	123	963253234	2011-04-13	138.75	138.75	0.00	3

```
(114 rows)
```

A SELECT statement that retrieves three columns from each row, sorted in descending sequence by invoice total

```
SELECT invoice_number, invoice_date, invoice_total
FROM invoices
ORDER BY invoice_total DESC
```

invoice_number	invoice_date	invoice_total
0-2058	2011-05-28	37966.19
P-0259	2011-07-19	26881.40
0-2060	2011-07-24	23517.58

```
(114 rows)
```

ORDER BY clause

The expanded syntax of the ORDER BY clause

```
ORDER BY expression [ASC|DESC][, expression [ASC|DESC]] ...
```

An ORDER BY clause that sorts by one column in ascending sequence

```
SELECT vendor_name,
  CONCAT(vendor_city, ', ', vendor_state, ' ', vendor_zip_code) AS address
FROM vendors
ORDER BY vendor_name
```

vendor_name	address
Abbey Office Furnishings	Fresno, CA 93722
American Booksellers Assoc	Tarrytown, NY 10591
American Express	Los Angeles, CA 90096
ASC Signs	Fresno, CA 93703

An ORDER BY clause that sorts by one column in descending sequence

```
SELECT vendor_name,
  CONCAT(vendor_city, ', ', vendor_state, ' ', vendor_zip_code) AS address
FROM vendors
ORDER BY vendor_name DESC
```

vendor_name	address
Zylka Design	Fresno, CA 93711
Zip Print & Copy Center	Fresno, CA 93777
Zee Medical Service Co	Washington, IA 52353
Yesmed, Inc	Fresno, CA 93718

SQL – SELECT (example)

A SELECT statement that returns all rows

```
SELECT vendor_city, vendor_state
FROM vendors
ORDER BY vendor_city
```

vendor_city	vendor_state
Anaheim	CA
Anaheim	CA
Ann Arbor	MI
Auburn Hills	MI
Boston	MA
Boston	MA
Boston	MA

```
(122 rows)
```

A SELECT statement that eliminates duplicate rows

```
SELECT DISTINCT vendor_city, vendor_state
FROM vendors
ORDER BY vendor_city
```

vendor_city	vendor_state
Anaheim	CA
Ann Arbor	MI
Auburn Hills	MI
Boston	MA
Brea	CA
Carol Stream	IL
Charlotte	NC

```
(53 rows)
```

LIMIT clause

- You can use the LIMIT clause to limit the number of rows returned by the SELECT statement.
    - This clause takes one or two integer arguments.
- If you code a single argument, it specifies the maximum row count, beginning with the first row.
- If you code both arguments, the offset specifies the first row to return, where the offset of the first row is 0.
    - If you want to retrieve all of the rows from a certain offset to the end of the result set, code -1 for the row count.

LIMIT clause

The expanded syntax of the LIMIT clause

```
LIMIT [offset,] row_count
```

A SELECT statement with a LIMIT clause that starts with the first row

```
SELECT vendor_id, invoice_total
FROM invoices
ORDER BY invoice_total DESC
LIMIT 5
```

vendor_id	invoice_total
110	37966.19
110	26881.40
110	23517.58
72	21842.00
110	20551.18

A SELECT statement with a LIMIT clause that starts with the third row

```
SELECT invoice_id, vendor_id, invoice_total
FROM invoices
ORDER BY invoice_id
LIMIT 2, 3
```

invoice_id	vendor_id	invoice_total
3	123	138.75
4	123	144.70
5	123	15.50

SQL – WHERE

- You can use a comparison operator to compare any two expressions.
  - If the result of a comparison is a true, the row being tested is included in the result set. If it's a false or null value, the row isn't included.

- MySQL automatically converts the data for comparison.
  - Character comparisons performed on MySQL databases are not case-sensitive.
    - E.g. 'CA' and 'ca' are considered equivalent.

- If you compare a null value using one of the comparison operators, the result is always a null value. To test for null values, use the IS NULL clause.

SQL – WHERE

- Find movies with rental rate of over $4

```
SELECT title, release_year, rental_rate FROM sakila.film WHERE rental_rate>4;
```

- Find customers that live in “Georgia”

```
SELECT * FROM sakila.address WHERE district = "Georgia";
```

- Find customers with first names starting with alphabet between A-D

```
SELECT customer_id, first_name, last_name FROM customer WHERE first_name<'E';
```

SQL – WHERE (IS NULL)

- A null value represents a value that’s unknown, unavailable, or not applicable.
    - It isn’t the same as a zero or an empty string ('').

The syntax of the WHERE clause with the IS NULL clause

```
WHERE expression IS [NOT] NULL
```

A SELECT statement that retrieves rows with null values

```
SELECT * FROM null_sample
WHERE invoice_total IS NULL
```

invoice_id	invoice_total
3	NULL

A SELECT statement that retrieves rows without null values

```
SELECT *
FROM null_sample
WHERE invoice_total IS NOT NULL
```

invoice_id	invoice_total
1	125.00
2	0.00
4	2199.99
5	0.00

SQL – WHERE (NULL)

- Get movie rentals that are not returned:

```
SELECT rental_id, customer_id, return_date
FROM rental
WHERE return_date IS NULL;
```

SQL – WHERE (example)

A SELECT statement that retrieves two columns and a calculated value for a specific invoice

```
SELECT invoice_id, invoice_total,
       credit_total + payment_total AS total_credits
FROM invoices
WHERE invoice_id = 17
```

invoice_id	invoice_total	total_credits
17	10.00	10.00

A SELECT statement that retrieves all invoices between given dates

```
SELECT invoice_number, invoice_date, invoice_total
FROM invoices
WHERE invoice_date BETWEEN '2011-06-01' AND '2011-06-30'
ORDER BY invoice_date
```

invoice_number	invoice_date	invoice_total
111-92R-10094	2011-06-01	19.67
989319-437	2011-06-01	2765.36
1-202-2978	2011-06-03	33.00

(37 rows)

A SELECT statement that returns an empty result set

```
SELECT invoice_number, invoice_date, invoice_total
FROM invoices
WHERE invoice_total > 50000
```

invoice_number	invoice_date	invoice_total

SQL – WHERE (example)

Examples of WHERE clauses that retrieve…

Vendors located in Iowa

```
WHERE vendor_state = 'IA'
```

Invoices with a balance due (two variations)

```
WHERE invoice_total - payment_total - credit_total > 0
WHERE invoice_total > payment_total + credit_total
```

Vendors with names from A to L

```
WHERE vendor_name < 'M'
```

Invoices on or before a specified date

```
WHERE invoice_date <= '2011-07-31'
```

Invoices on or after a specified date

```
WHERE invoice_date >= '2011-07-01'
```

Invoices with credits that don't equal zero (two variations)

```
WHERE credit_total <> 0
WHERE credit_total != 0
```

SQL - LIKE and REGEXP

- You use the LIKE and REGEXP operators to retrieve rows that match a string pattern, called a mask. The mask determines which values in the column satisfy the condition.

- The mask for a LIKE phrase can contain special symbols, called wildcards. The mask for a REGEXP phrase can contain special characters and constructs. Masks aren’t case-sensitive.

- Most LIKE and REGEXP phrases significantly degrade performance compared to other types of searches, so use them only when necessary.

SQL - LIKE and REGEXP

- LIKE wildcards

Symbol	Description
%	Matches any string of zero or more characters
_	Matches any single character

- REGEXP special characters and constructs

Construct	Description
^	Matches the pattern to the beginning of the value being tested
$	Matches the pattern to the end of the value being tested
.	Matches any single character
[TEXTlist]	Matches any single character listed within the brackets
[TEXT1-TEXT2]	Matches any single character within the given range
|	Separates two string patterns and matches either one

SQL - LIKE and REGEXP (Examples)

The syntax of the WHERE clause with a LIKE phrase

```
WHERE match_expression [NOT] LIKE pattern
```

The syntax of the WHERE clause with a REGEXP phrase

```
WHERE match_expression [NOT] REGEXP pattern
```

WHERE clauses that use the LIKE and REGEXP operators

Example	Results that match the mask
`WHERE vendor_city LIKE 'SAN%'`	"San Diego", "Santa Ana"
`WHERE vendor_name LIKE 'COMPU_ER%'`	"Compuserve", "Computerworld"
`WHERE vendor_city REGEXP 'SA'`	"Pasadena", "Santa Ana"
`WHERE vendor_city REGEXP '^SA'`	"Santa Ana", "Sacramento"
`WHERE vendor_city REGEXP 'NA$'`	"Gardena", "Pasadena", "Santa Ana"
`WHERE vendor_city REGEXP 'RS|SN'`	"Traverse City", "Fresno"
`WHERE vendor_state REGEXP 'N[CV]'`	"NC" and "NV" but not "NJ" or "NY"
`WHERE vendor_state REGEXP 'N[A-J]'`	"NC" and "NJ" but not "NV" or "NY"
`WHERE vendor_contact_last_name REGEXP 'DAMI[EO]N'`	"Damien" and "Damion"
`WHERE vendor_city REGEXP '[A-Z][AEIOU]N$'`	"Boston", "Mclean", "Oberlin"

Logical operators: AND, OR, NOT

- You can use the AND and OR logical operators to create compound conditions that consist of two or more conditions.
    - You use the AND operator to specify that the search must satisfy both of the conditions, and you use the OR operator to specify that the search must satisfy at least one of the conditions.
- You can use the NOT operator to negate a condition.
- When MySQL evaluates a compound condition, it evaluates the operators in this sequence: (1) NOT, (2) AND, and (3) OR.
    - You can use parentheses to override this order of precedence or to clarify the sequence in which the operations are evaluated.

Logical operators: examples

The syntax of the WHERE clause with logical operators

```
WHERE [NOT] search_condition_1 {AND|OR} [NOT] search_condition_2 ...
```

Examples of WHERE clauses that use logical operators

The AND operator

```
WHERE vendor_state = 'NJ' AND vendor_city = 'Springfield'
```

The OR operator

```
WHERE vendor_state = 'NJ' OR vendor_city = 'Pittsburg'
```

The NOT operator

```
WHERE NOT vendor_state = 'CA'
```

The NOT operator in a complex search condition

```
WHERE NOT (invoice_total >= 5000 OR NOT invoice_date <= '2011-08-01')
```

The same condition rephrased to eliminate the NOT operator

```
WHERE invoice_total < 5000 AND invoice_date <= '2011-08-01'
```

IN operator

- You can use the IN phrase to test whether an expression is equal to a value in a list of expressions.
    - Each of the expressions in the list is automatically converted to the same type of data as the test expression.
    - The list of expressions can be coded in any order without affecting the order of the rows in the result set.
- You can use the NOT operator to test for an expression that’s not in the list of expressions.
- You can also compare the test expression to the items in a list returned by a subquery.

IN operator: examples

The syntax of the WHERE clause with an IN phrase

```
WHERE test_expression [NOT] IN
      ({subquery|expression_1 [, expression_2]...})
```

Examples of the IN phrase

An IN phrase with a list of numeric literals

```
WHERE terms_id IN (1, 3, 4)
```

An IN phrase preceded by NOT

```
WHERE vendor_state NOT IN ('CA', 'NV', 'OR')
```

An IN phrase with a subquery

```
WHERE vendor_id IN
    (SELECT vendor_id
     FROM invoices
     WHERE invoice_date = '2011-07-18')
```

- Sakila:
- SELECT * FROM film WHERE length IN (120, 121);
- SELECT * FROM film WHERE rental_rate NOT IN (2.99, 0.99);

BETWEEN operator

- You can use the BETWEEN phrase to test whether an expression falls within a range of values.
  - The lower limit must be coded as the first expression, and the upper limit must be coded as the second expression.
  - Otherwise, MySQL returns an empty result set.
- The two expressions used in the BETWEEN phrase for the range of values are inclusive.
  - That is, the result set includes values that are equal to the upper or lower limit.
- You can use the NOT operator to test for an expression that's not within the given range.

BETWEEN operator: examples

The syntax of the WHERE clause with a BETWEEN phrase

```
WHERE test_expression [NOT] BETWEEN begin_expression AND end_expression
```

Examples of the BETWEEN phrase

A BETWEEN phrase with literal values

```
WHERE invoice_date BETWEEN '2011-06-01' AND '2011-06-30'
```

A BETWEEN phrase preceded by NOT

```
WHERE vendor_zip_code NOT BETWEEN 93600 AND 93799
```

A BETWEEN phrase with a test expression coded as a calculated value

```
WHERE invoice_total - payment_total - credit_total BETWEEN 200 AND 500
```

A BETWEEN phrase with the upper and lower limits coded as calculated values

```
WHERE payment_total BETWEEN credit_total AND credit_total + 500
```

- Sakila:
- SELECT * FROM film WHERE length BETWEEN 120 AND 125;
- SELECT * FROM film WHERE title BETWEEN "AA" AND "AE";

SQL – CAST

- CAST(value AS ***datatype***)

- Example:

```
SELECT
CAST('2021-03-24' AS DATE) today_date,
CAST('10:00:00' AS TIME) today_date
```

```
SELECT rental_date, cast(rental_date AS DATE) AS TD FROM rental;
SELECT rental_date, cast(rental_date AS YEAR) AS TD FROM rental;
SELECT rental_date, cast(rental_date AS CHAR(7)) AS TD FROM rental;
```

datatype	Description
DATE	Converts *value* to DATE. Format: "YYYY-MM-DD"
DATETIME	Converts *value* to DATETIME. Format: "YYYY-MM-DD HH:MM:SS"
DECIMAL	Converts *value* to DECIMAL. Use the optional M and D parameters to specify the maximum number of digits (M) and the number of digits following the decimal point (D).
TIME	Converts *value* to TIME. Format: "HH:MM:SS"
CHAR	Converts *value* to CHAR (a fixed length string)
NCHAR	Converts *value* to NCHAR (like CHAR, but produces a string with the national character set)
SIGNED	Converts *value* to SIGNED (a signed 64-bit integer)
UNSIGNED	Converts *value* to UNSIGNED (an unsigned 64-bit integer)
BINARY	Converts *value* to BINARY (a binary string)

SQL – CASE (logical expression)

- Get customers with their status (active/inactive):

```
SELECT c.first_name, c.last_name,
CASE
    WHEN active = 1
    THEN "ACTIVE"
    ELSE "INACTIVE"
END active_type
FROM customer AS c;
```

SQL – CASE (logical expression)

- Get number of rentals for the months of May, June, July:

```
SELECT monthname(rental_date) rental_month, count(*) num_rentals
FROM rental
WHERE rental_date BETWEEN "2005-05-01" AND "2005-08-01"
GROUP BY monthname(rental_date);
```

```
SELECT
     SUM(CASE WHEN monthname(rental_date) = "May" THEN 1 ELSE 0 END) May_rentals,
     SUM(CASE WHEN monthname(rental_date) = "June" THEN 1 ELSE 0 END) June_rentals,
     SUM(CASE WHEN monthname(rental_date) = "July" THEN 1 ELSE 0 END) July_rentals
FROM rental
WHERE rental_date BETWEEN "2005-05-01" AND "2005-08-01";
```

SELECT: built-in functions

- An expression can include any of the functions that are supported by MySQL. A function performs an operation and returns a value.

- To use a function, code the function name followed by a set of parentheses. Within the parentheses, code any parameters (or arguments) required by the function.
    - If a function requires two or more arguments, separate them with commas.

- To code a literal value for a string, enclose one or more characters within single quotes (') or double quotes (").

- Complete list of functions:
    - https://dev.mysql.com/doc/refman/8.0/en/built-in-function-reference.html

SELECT: built-in functions

- Select full name of actors in one column

```
select CONCAT(first_name, " ", last_name) AS full_name
from actor
ORDER BY last_name DESC, first_name DESC
LIMIT 10;
```

SELECT: built-in functions

A SELECT statement that uses the LEFT function

```
SELECT vendor_contact_first_name, vendor_contact_last_name,
       CONCAT(LEFT(vendor_contact_first_name, 1),
              LEFT(vendor_contact_last_name, 1)) AS initials
FROM vendors
```

vendor_contact_first_name	vendor_contact_last_name	initials
Francesco	Alberto	FA
Ania	Irvin	AI
Lukas	Liana	LL

```
(122 rows)
```

A SELECT statement that uses the DATE_FORMAT function

```
SELECT invoice_date,
       DATE_FORMAT(invoice_date, '%m/%d/%y') AS 'MM/DD/YY',
       DATE_FORMAT(invoice_date, '%e-%b-%Y') AS 'DD-Mon-YYYY'
FROM invoices
```

invoice_date	MM/DD/YY	DD-Mon-YYYY
2011-04-08	04/08/11	8-Apr-2011
2011-04-10	04/10/11	10-Apr-2011
2011-04-13	04/13/11	13-Apr-2011

```
(114 rows)
```

A SELECT statement that uses the ROUND function

```
SELECT invoice_date, invoice_total,
       ROUND(invoice_total) AS nearest_dollar,
       ROUND(invoice_total, 1) AS nearest_dime
FROM invoices
```

invoice_date	invoice_total	nearest_dollar	nearest_dime
2011-04-08	3813.33	3813	3813.3
2011-04-10	40.20	40	40.2
2011-04-13	138.75	139	138.8

```
(114 rows)
```

Before Next Class

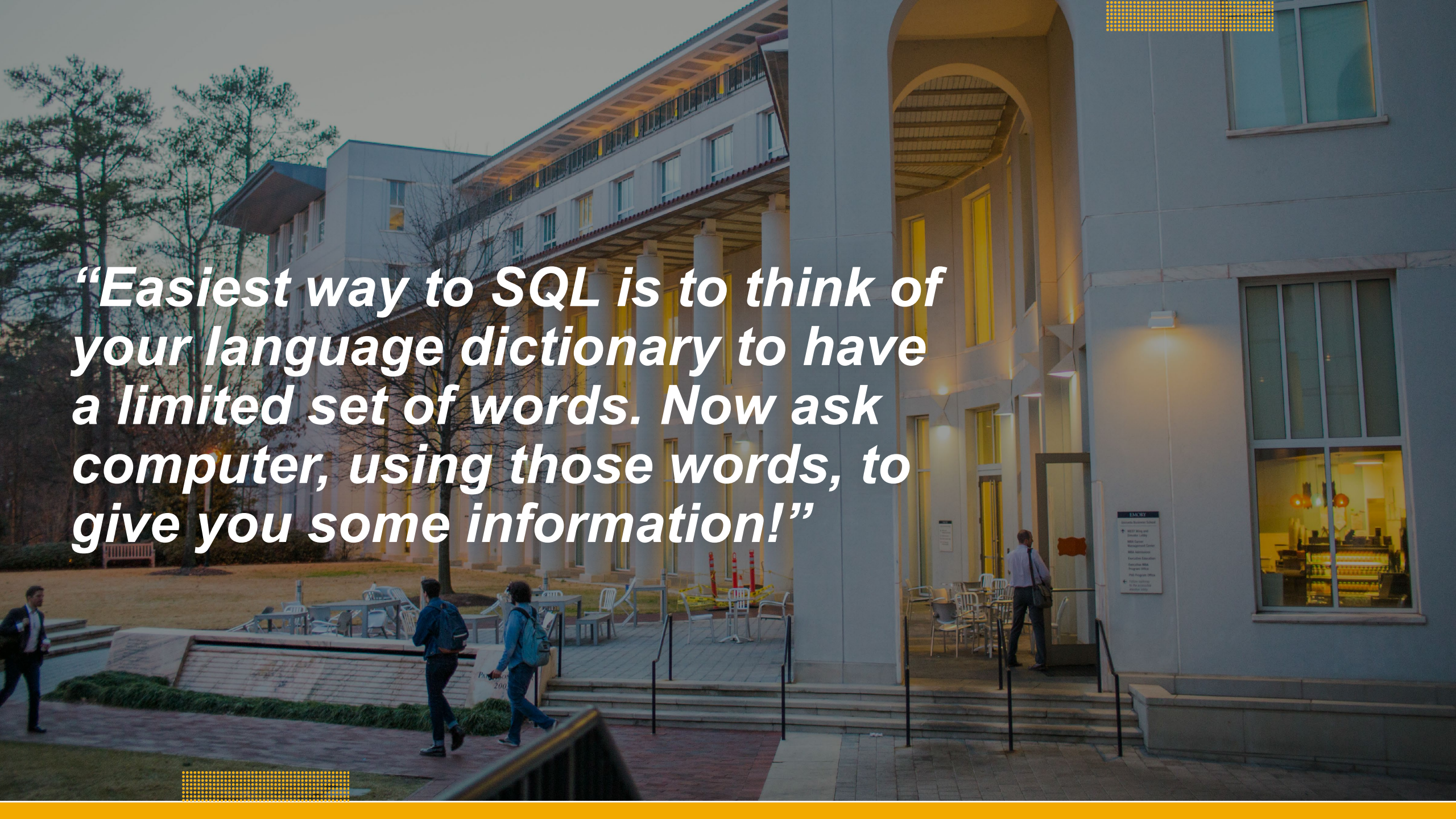
- Assignment 1

- Practice the queries covered in class today (Most of this was already covered in bootcamp)
  - Use coffee shop and sakila data:
    - Download the coffee shop data from canvas
    - Load the csv files
    - Create PKs and FKs
    - Explore the data with SELECT queries

Reflect

Close your eyes, relax, and reflect on ONE take away from today's class.

5:00


“Easiest way to SQL is to think of your language dictionary to have a limited set of words. Now ask computer, using those words, to give you some information!”